Day 1 Implementation — Telemetry Analyser

Goal

Complete the starter program so it reports minimum, maximum, average, and a simple status from distance readings in metres.

Step 1 — Inspect before editing

Open `starter/telemetry_analyser.cpp`. Identify:

- the input vector;
- the unfinished function declarations;
- the output section in `main`.

Compile the starter once. Record the compiler output.

Step 2 — Implement `minimum_reading`

Contract:

- Input: `const std::vector<double>& readings`.
- Return: smallest value.
- If empty, return `0.0` (the caller will separately report that no readings exist).

Hint: initialise the candidate minimum from the first element only after checking the vector is not empty.

Step 3 — Implement `maximum_reading`

Use the same pattern as the minimum function, but update when a larger value is found.

Step 4 — Implement `average_reading`

Accumulate all readings and divide by the number of elements. Return `0.0` for an empty vector. Do not divide by zero.

Step 5 — Implement `classify_robot`

Use these rules:

- `CRITICAL`: minimum distance is less than `0.20 m`.
- `CAUTION`: minimum distance is at least `0.20 m` and less than `0.50 m`.

- `CLEAR`: minimum distance is at least `0.50 m`.

Step 6 — Build and test

```
cd starter
g++ -std=c++17 -Wall -Wextra -Wpedantic telemetry_analyser.cpp -o telemetry
./telemetry
```

Expected output for the supplied readings:

- Minimum: `0.18`
- Maximum: `1.42`
- Average: approximately `0.70`
- Status: `CRITICAL`

Formatting can differ, but the values and classification must be correct. The program finishes with self-checks and should end with `Day 1 telemetry analyser: 8/8 checks passed`. Any `FAIL` line names the check that is still wrong.

Step 7 — Explain your design

Without reading the file, explain:

1. Why are the readings passed by `const` reference?
2. Why is the empty-vector case checked?
3. What would happen if the loop started at index 1 instead of 0?